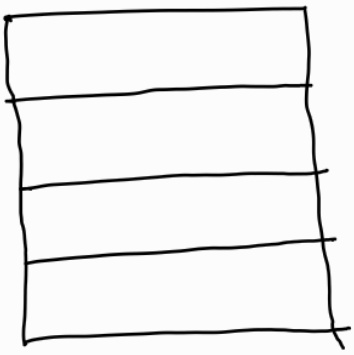
→



→ forward pass :—

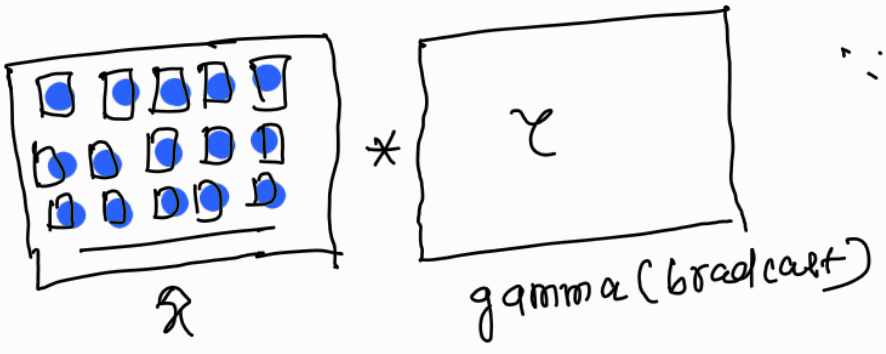
$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i, \quad \sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

Normalize Each Vector :—

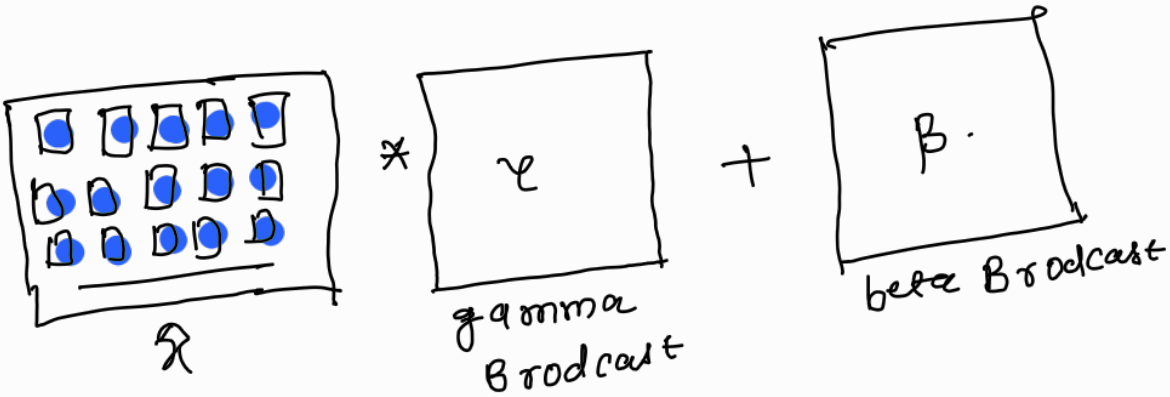


→



$\because y_i = \gamma \hat{x}_i + \beta$

→



$$\because y_i = \gamma \hat{x}_i + \beta$$

→ Backward pass :-

$$y_i = \gamma \hat{x}_i + \beta$$

→ Upstream grad :-

$$\frac{\partial z}{\partial y_i} = g_i$$

→ what we need ?

$$\frac{\partial z}{\partial \gamma} * \frac{\partial y}{\partial \beta} * \frac{\partial z}{\partial x} \quad \because y = \gamma \hat{x}_i + \beta$$

$$\frac{\partial z}{\partial \gamma} = \sum_{i=1}^{N} g_i \hat{x}_i \,, \quad \frac{\partial z}{\partial \beta} = \sum_{i=1}^{N} g_i$$

$$\frac{\partial z}{\partial x} = \frac{1}{\sqrt{\sigma^2 + \epsilon}} \left[d\hat{x} - \text{mean}(d\hat{x}) - \hat{x} \cdot \text{mean}(d\hat{x} \odot \hat{x}) \right]$$

$$\frac{\partial z}{\partial \hat{x}_i} = \frac{\partial z}{\partial y_i} \cdot \frac{\partial y_i}{\partial \hat{x}_i}$$

→ details of Layer Norm: —

$$y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}} * \gamma + \beta.$$

Data: —

$d \in B \times S \times E$ ← for 2 LM

normalize along the last dim E

$$\mu_{b,s} = \frac{1}{E} \sum_{e=1}^{E} x_{b,s,e}.$$

$$Var_{b,s} = \frac{1}{E} \sum_{e=1}^{E} (x_{b,s,e} - \mu_{b,s})^2$$

$$\hat{x}_{b,s,e} = \frac{x_{b,s,e} - \mu_{b,s}}{\sqrt{var_{b,s} + \epsilon}}$$

$$y_{b,s,e} = \gamma_e \cdot \hat{x}_{b,s,e} + \beta_e$$

→ Given upstream grand: — $\frac{\partial z}{\partial y_{b,s,e}}$ we need

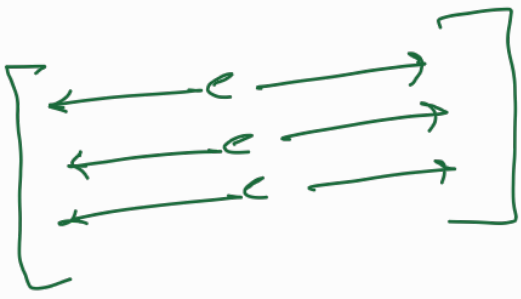
$$\frac{\partial h}{\partial \gamma_e}, \frac{\partial z}{\partial \beta_e}, \frac{\partial z}{\partial x_{b,s,e}}.$$

Path:-

→ $x \longrightarrow \hat{x} \longrightarrow \hat{x}\gamma \longrightarrow \hat{x}\gamma + \beta$.

→ $x \in (B, S, e)$

$x \in (\beta x S, e)$



→ Remember our B,S, index are independent ∴ the good are accumulated. for simplicity I will ignore them.

$$\mu = \frac{1}{E}\sum_{k=1}^{E} x_k \;, \quad \hat{\mu} = x_k - \mu \leftarrow \text{mean centered.}$$

$$\sigma^2 = \frac{1}{E}\sum_{k=1}^{E} \mu_k^2$$

$$S = \sqrt{\sigma^2 + \epsilon}$$

$$\hat{x}_j = \frac{\mu_j}{S} = \frac{x_j - \mu}{S}$$

$$\boxed{y_i = \gamma_j \hat{x}_j + \beta_j} \longrightarrow (A)$$

Vector:-

$[x_1\ x_2\ x_3]$, μ, σ^2 = ?

$$\Rightarrow \left[\frac{x_1-\mu}{\sigma^2}, \frac{x_2-\mu}{\sigma^2}, \frac{x_3-\mu}{\sigma^2}\right]$$

$$\Rightarrow [\hat{x}_1, \hat{x}_2, \hat{x}_3]$$

)

shift by γ.

$\gamma \in [\gamma_1, \gamma_2, \gamma_3]$

$\beta \in [\beta_1, \beta_2, \beta_3]$

$\Rightarrow [\hat{x}_1 \cdot \gamma_1 + \beta_1, \ \hat{x}_2 \cdot \gamma_2 + \beta_2, \ \hat{x}_3 \cdot \gamma_3 + \beta_3]$

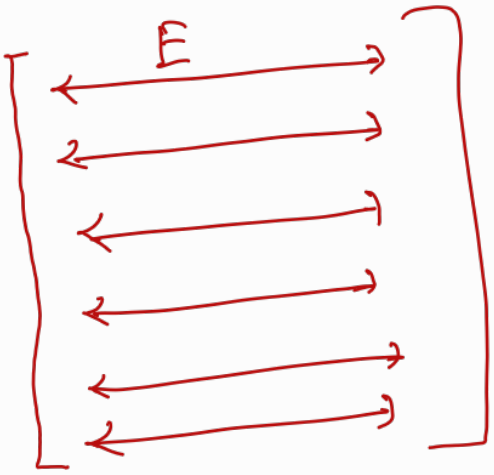
$$y_j = [\hat{x}_j \cdot \gamma_j + \beta_j] \quad \text{— (A)}$$

$$y = \sum_{j=1}^{E} (\hat{x}_j \cdot \gamma_j + \beta_j)$$

upstream gradient

$$\nabla_j = \frac{dz}{dy_j} \leftarrow \text{upstream gradient}$$

$$\frac{\partial z}{\partial x_i} = \sum_{j=1}^{E} \frac{dz}{dy_j} \cdot \frac{dy_j}{dx_i} \leftarrow \text{sum up all those conti—}$$



upstream gradient

$$\frac{\partial z}{\partial x_i} = \sum_{j=1}^{E} \nabla_j \underbrace{\frac{dy_j}{d\hat{x}_j}}_{\gamma_j} \cdot \frac{d\hat{x}_j}{dx_i}$$

$\because y = \gamma \hat{x} + b$

$$\frac{dy}{d\hat{x}} = \gamma \frac{d\hat{x}}{d\hat{x}} + 0$$

$$\boxed{\frac{dy}{d\hat{x}} = \gamma}$$

$$\frac{\partial \eta}{dx_i} = \sum_{j=1}^{E} \nabla_j \gamma_j \cdot \frac{dx_j}{dx_i}$$

Solve this. →

$$\rightarrow \mu = \frac{1}{E} \sum_{k=1}^{E} x_k \rightarrow \frac{d\mu}{dx_i} = \frac{1}{E}$$

→ Ex:- $\mu = \frac{1}{E}[x_1, x_2 \text{ ---- } x_E]$

$\frac{d}{dx}$

$$\frac{\partial u}{\partial x} = \left[\frac{\partial u}{dx_1}, \frac{\partial u}{dx_2}, \frac{\partial u}{dx_3}, \frac{\partial u}{dx_4}\right]$$

$\because \frac{\partial x_1}{\partial x_1} = 1, \quad \frac{\partial x_2}{\partial x_1} = 0, \quad \frac{\partial x_3}{\partial x_1} = 0$

means $\frac{\partial x_i}{\partial x_i} = 1$

$$\frac{\partial u}{\partial x_i} = \frac{\partial}{\partial x_i}\left(\frac{1}{E}(x_1+x_2+x_3+x_4)\right)$$

$i=1$ $\qquad$ $i=k$

$$\frac{\partial u}{\partial x_1} = \left(\frac{1}{E}(1+0+0+0)\right) \qquad \frac{\partial u}{\partial x_k} = \frac{1}{E}(0+0+\dots)$$

$$\frac{\partial u}{\partial x_1} = \left(\frac{1}{E}\right) \qquad \frac{\partial u}{\partial x_k} = \left(\frac{1}{E}\right)$$

So

$$\boxed{\frac{\partial u}{\partial x_i} = \frac{1}{E}}$$

$$u = \frac{1}{E}\sum_{k=1}^{E} x_k \longrightarrow \frac{\partial u}{\partial x_i} = \frac{1}{E}$$

$$u_j = x_j - u$$

$$\frac{\partial}{\partial x_i} \longrightarrow \frac{\partial u_j}{\partial x_i} = \frac{\partial}{\partial x_i}(x_j - u)$$

$$= \frac{\partial x_j}{\partial x_i} - \frac{\partial u}{\partial x_i}$$

$\frac{\partial u}{\partial x_i} \rightarrow \frac{1}{E}$

$\frac{\partial x_j}{\partial x_i}$:

if $i = j \rightarrow 1$

if $i \neq j \rightarrow 0$

$$\frac{\partial u_j^o}{\partial x_i^o} = \frac{\partial x_j^o}{\partial x_i^o} - \frac{1}{E}$$

$$\delta = \frac{\partial x_j^o}{\partial x_i^o} = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases}$$

$$\boxed{\frac{\partial u_j^o}{\partial x_i} = \delta - \frac{1}{E}} \leftarrow A$$

→ Now about σ^2 :-

$$\sigma^2 = \frac{1}{E} \sum_{k=1}^{E} u_k^2$$

$$\frac{\partial \sigma^2}{\partial x_i^o} = \frac{\partial}{\partial x_i^o} \left(\frac{1}{E} \sum_{k=1}^{E} u_k^2 \right)$$

$$= \left(\frac{1}{E}\right) \left(\frac{\partial}{\partial x_i^o} \left[u_1^2 + u_2^2 + \cdots u_E^2 \right] \right)$$

$$= \left(\frac{1}{E}\right) \left\{ \frac{\partial}{\partial x_i^o} \begin{array}{l} \xrightarrow{i=1} 2u_1 + 0 + 0 \cdots \\ \xrightarrow{i=2} 0 + 2u_2 + \cdots \\ \searrow^{i=k} 0 + 0 + 0 \cdots 2u_{k+00} \end{array} \right\}$$

$$= \left(\frac{1}{E}\right) \left[\sum_{k=1}^{E} 2u_k \cdot \underbrace{\frac{\partial u_k}{\partial x_i^o}}_{A} \right]$$

$$= \left(\frac{2}{E}\right)\left(\sum_{k=1}^{E} \mu_k\left(\delta_{i,k} - \frac{1}{E}\right)\right)$$

$$= \left(\frac{2}{E}\right)\left[\sum_{k=1}^{E}\left(\mu_k \cdot \delta_{i,k} - \frac{\mu_k}{E}\right)\right]$$

$$= \left(\frac{2}{E}\right)\left(\sum_{k=1}^{E} \mu_k \cdot \delta_{i,k} - \sum_{k=1}^{E}\frac{\mu_k}{E}\right)$$

$\mu_k \cdot \delta_{i,k}$ → $\begin{cases} 0 \text{ if } i \neq k \\ 1 \text{ if } i = k \end{cases}$

$$= \left(\frac{2}{E}\right)\left[\begin{cases} 0 \text{ if } i \neq k \\ 1 \text{ if } i = k \end{cases} - \sum_{k=1}^{E}\frac{\mu_k}{E}\right]$$

$$= \left(\frac{2}{E}\right)\left[\mu_i - \underbrace{\frac{1}{E}\sum_{k=1}^{E}\mu_k}\right]$$

$$= \sum_{k=1}^{k} \mu_k \delta_{ik}$$

$\Rightarrow \mu_1\delta_{1k} + \mu_2\delta_{2k} + \mu_3\delta_{3k} + \cdots$

$\Rightarrow$ $k = 2$

then

$\mu_1\delta_{1k} + \mu_2\delta_{2k} + \mu_3\delta_{3k} + \cdots$

$\delta_{1k} \to 0$

$\mu_k = x - \mu$

→ it's mean center Data

then what is Avg?

$\frac{1}{E}\sum_{k=1}^{E}\mu_k$

$\frac{1}{E}\sum_{k=1}^{E}(x_k - \mu)$

→ mean centred data so mean of mean is zero.

$$= \left(\frac{2}{E}\right)\left[\mu_i - 0\right]$$

$$= \left(\frac{2}{E}\right)(\mu_i)$$

$$\boxed{\frac{\partial \sigma^2}{\partial x_e} = \frac{2\mu_e}{E}}$$

$$S = \sqrt{\sigma^2 + \epsilon} = (\sigma^2 + \epsilon)^{1/2}$$

$$\frac{\partial S}{\partial x_i} = \frac{1}{2}(\sigma^2 + \epsilon)^{-1/2} \cdot \frac{d(\sigma^2 + \epsilon)}{dx_i}$$

$$= \frac{1}{2}(\sigma^2 + \epsilon)^{-1/2} \cdot \left[\frac{d\sigma^2}{dx_i} + \frac{\partial \epsilon}{\partial x_i}\right]$$

$$= \frac{1}{2}(\sigma^2 + \epsilon)^{-1/2} \cdot \left[\frac{d\sigma^2}{dx_i}\right]$$

$$\boxed{\frac{\partial S}{\partial x_i} = \frac{1}{2\sqrt{\sigma^2 + \epsilon}} * \frac{2\mu_i}{E}}$$

$\mu = x_k - \mu$ ← Derivative (✓)

$\sigma^2 = \frac{1}{E}\sum_{k=1}^{E} \mu_k^2$ ← Derivative (✓)

$s = \sqrt{\sigma^2 + \epsilon}$ ← Derivative (✓)

$\hat{x}_j = \frac{\mu_j}{s} = \frac{x_j - \mu}{s}$ ← Now this.

$y_j = \gamma_j \hat{x}_j + \beta_j$

$$\Rightarrow \hat{x}_j = \frac{\mu_j}{s} = \frac{x_j - \mu}{s}$$

derivative of $\hat{x}_j$:-

$$\frac{\partial \hat{x}_j}{\partial x_i} = \frac{\partial}{\partial x_i}\left(\frac{\mu_j}{s}\right) = \mu_j \frac{d}{dx_i}\left(\frac{1}{s}\right) + \left(\frac{1}{s}\right)\frac{\partial}{\partial x_i}(\mu_j)$$

$$= \underbrace{\mu_j \frac{d}{dx_i}\left(\frac{1}{s}\right)}_{①} + \underbrace{\left(\frac{1}{s}\right)\left(\frac{\partial}{\partial x_i}(\mu_j)\right)}_{②}$$

$$= -\frac{\mu_j}{s^2}\frac{ds}{dx_j} + \left(\frac{1}{s}\right)\left(\frac{\partial \mu_j}{\partial x_i}\right)$$

$$= \frac{1}{2\sqrt{\sigma^2+\epsilon}} \frac{2\mu_i}{E} \qquad \left(\delta_{ij} - \frac{1}{E}\right)$$

$$\Rightarrow \frac{d}{dx} x^{+}$$

$$\Rightarrow (-1)\, x^{-1+}$$

$$\Rightarrow (-1)\cdot x^{-2}$$

$$= \left(-1/x^2\right)\frac{ds}{dx}$$

$$= \frac{-u_j \cdot 2u_i}{s^2 \cdot 2\sqrt{\sigma^2+\epsilon}\,E} + \left(\frac{1}{s}\right)\left(\delta_{ij} - \frac{1}{E}\right)$$

$$= \frac{-2 \cdot u_j \cdot u_i}{2\sqrt{\sigma^2+\epsilon}\cdot E \cdot s^2} + \left(\frac{1}{s}\right)\left(\delta_{ij} - \frac{1}{E}\right)$$

$$\sqrt{\sigma^2+\epsilon} \longrightarrow s$$

$$= \frac{-u_j \cdot u_i}{s^3 \cdot E} + \left(\frac{1}{s}\right)\left(\delta_{ij} - \frac{1}{E}\right)$$

$$= \left(\frac{1}{s}\right)\left[\frac{-u_j \cdot u_i}{s^2 E} + \delta_{ij} - \frac{1}{E}\right]$$

$$= \left(\frac{1}{s}\right)\left[\delta_{ij} - \frac{1}{E} - \frac{u_j u_i}{s\ s} \cdot \frac{1}{E}\right]$$

$$\frac{u}{s} = \hat{x}$$

$$\frac{\partial \hat{x}_j}{\partial x_i} = \left(\frac{1}{s}\right)\left[\delta_{i,j} - \frac{1}{E} - \hat{x}_j \cdot \hat{x}_i \cdot \frac{1}{E}\right]$$

Now remember that:—

this is solve.

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{E} \nabla_j \gamma_j \frac{\delta \hat{x}_j}{\delta x_i}$$

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{E} \nabla_j \gamma_j \left\{ \frac{1}{s}\left[\delta_{i,j} - \frac{1}{E} - \hat{x}_j \cdot \hat{x}_i \cdot \frac{1}{E}\right]\right\}$$

$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{E} \nabla_j \delta_j \left\{ \frac{1}{s}\left[\delta_{i,j} - \frac{1}{E} - \frac{\hat{x}_j \cdot \hat{x}_i}{E}\right]\right\}$$

$$= \frac{1}{s}\left[\sum_{j=1}^{E} \nabla_j \cdot s_j \left\{ \delta_{i,j} - \frac{1}{E} - \frac{\hat{x}_j \cdot \hat{x}_i}{E}\right]\right]$$

$$= \frac{1}{s}\left[\sum_{j=1}^{E} \nabla_j \cdot s_j \cdot \delta_{i,j} - \sum_{j=1}^{E} \nabla_j s_j \cdot \frac{1}{E} - \frac{1}{E}\sum_{j=1}^{E} \nabla_j s_j \cdot \hat{x}_j \cdot \hat{x}_i\right]$$

$\delta_{i,j}$ ↓

$$\begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

$$=$$

$$\frac{\partial z}{\partial x_i} = \frac{1}{s}\left[\nabla_i \gamma_i - \frac{1}{E}\sum_{j=1}^{E} \nabla_j \cdot \gamma_j - \frac{\hat{x}_i}{E}\sum_{j=1}^{E} \nabla_j \gamma_j \hat{x}_j\right]$$

Looks like average.

$$\rightarrow \nabla = \frac{\partial z}{\partial y} \text{ and } \frac{\partial y}{d\hat{x}} = \gamma$$

so for simplicity, our upstream grand to $\hat{x}$ (not just y) is

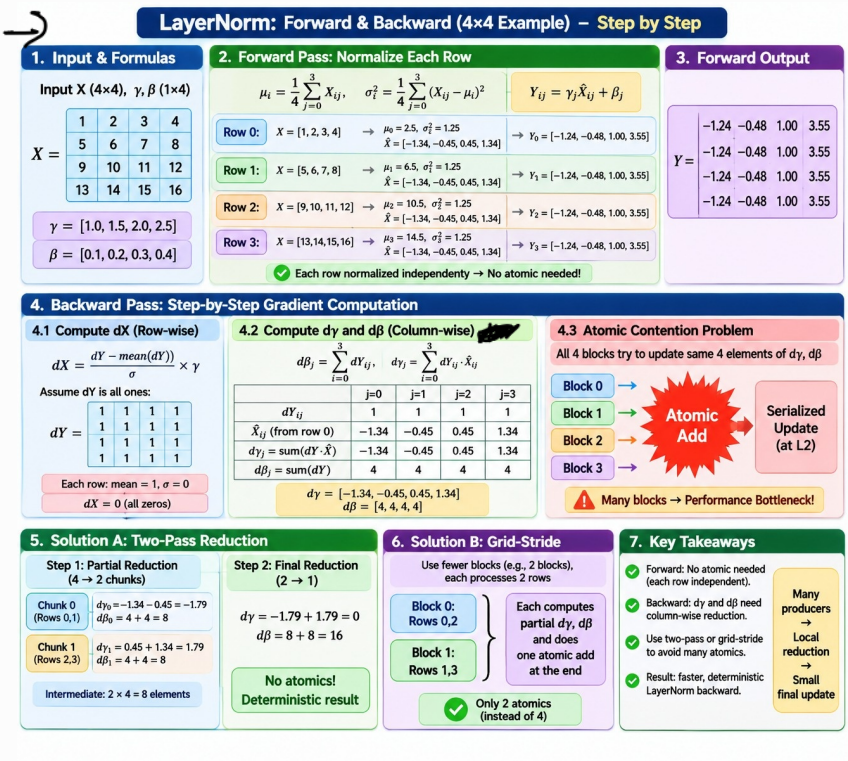
$$\frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial \hat{x}} = \nabla\gamma \triangleq \tilde{\nabla}$$ ← let just give it new

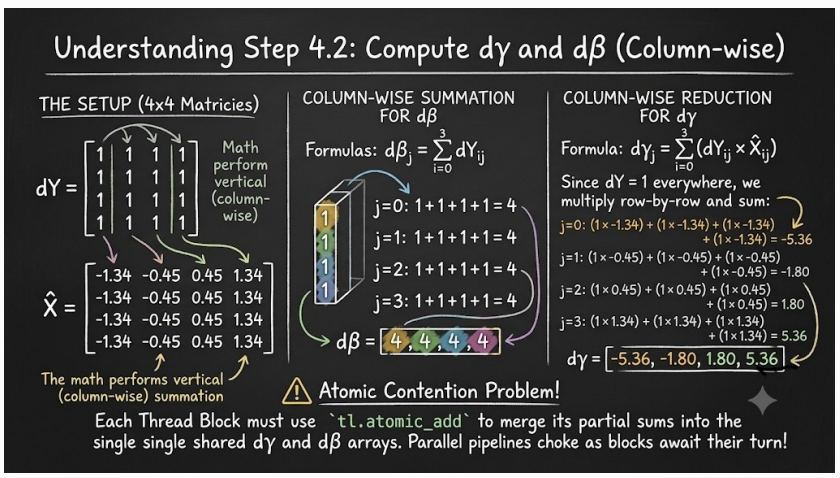
$$\frac{\partial z}{\partial x_i} = \frac{1}{s}\left[\tilde{\nabla} - \underbrace{\frac{1}{E}\sum_{j=1}^{E}\tilde{\nabla}}_{\text{mean}} - \frac{\hat{x}_i}{E}\underbrace{\sum_{j=1}^{E}\tilde{\nabla}\cdot\hat{x}_j}_{\text{mean}}\right]$$

$$\frac{\delta z}{\delta x_i} = \frac{1}{\sqrt{\sigma^2+\epsilon}}\left[\tilde{\nabla} - \text{mean}(\tilde{\nabla}) - \hat{x}\cdot\text{mean}(\tilde{\nabla}\cdot\hat{x})\right]$$

$$\frac{\partial z}{\partial x} = \frac{1}{\sqrt{\sigma^2+\epsilon}}\left[d\hat{z} - \text{mean}(d\hat{x}) - \hat{x}\cdot\text{mean}(d\hat{x}\odot\hat{x})\right]$$

$$\frac{\partial z}{\partial \hat{x}_i} = \frac{\partial z}{\partial y_i}\cdot\frac{\partial y_i}{\partial \hat{x}_i}$$





← Atomic add problem during Back-prop

↳ colᵐ wise - sum for dβ.

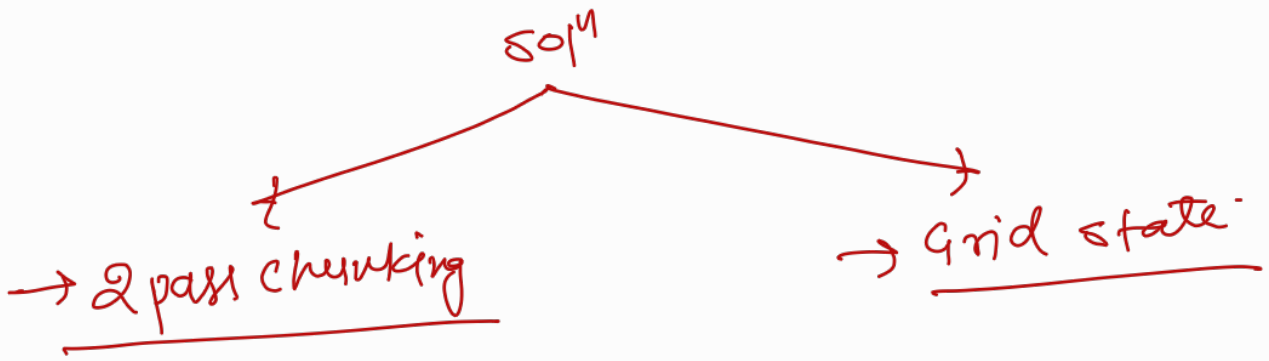
colᵐ wise Reduction:-

$$d\gamma_j = \sum_{i=0}^{3} (dY_{i,j} \times \hat{X}_{i,j})$$

$d\gamma = 1 \rightarrow \hat{X} =$ [−1.34, −1.34, −1.34, −1.34]

$\times$ [1 1 1 1; 1 1 1 1; 1 1 1 1; 1 1 1 1] → $d\gamma_1 = -5.36$, $d\gamma_2 =$ ____, $d\gamma_3 =$ ____, $d\gamma_4 =$ ____

$d\gamma_1, d\gamma_2, d\gamma_3, d\gamma_4 \rightarrow \Sigma = d\gamma$ ← for $d\gamma$ need all $[d\gamma_1, d\gamma_2, d\gamma_3, d\gamma_4]$



Method 01 :—

```
def layer_norm(x, weight, bias, eps):
    *dim, e_dim = x.shape
    if len(dim) > 1:
        x_reshaped = x.reshape(-1, e_dim)
    else:
        x_reshaped = x

    mean = x_reshaped.mean(axis=-1, keepdim=True)   ← 1st kernal
    var = x_____.var(_____)                         ← 2nd kernal

    normed = (x_resh - mean) / cp.sqrt(var + eps)
                  3                  4      5

    # X_NXE * W(IXE)
    out = normed * weight.reshape(1,-1)             ← 6th kernal
    if bias is not None:
        out = out + bias.reshape(1,-1)              ← 7th kernal
    if len(dim) > 1:
        out = out.reshaped(*dims, embed_dim)        ← 8th kernal

    return out
```

→ total 8 kernal launched convert into → 1 kernal launched

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y = \alpha \hat{x} + \beta.$$

$\mu \leftarrow$ mean() → (1)
$\sigma^2 \leftarrow$ var() → (2)
$\delta \leftarrow (\sigma^2 + \epsilon)$ → (3)
$s \leftarrow \sqrt{s}$ → (4)
$\hat{x} \leftarrow (x - \mu)$ → (5)
$\hat{x} \leftarrow \hat{x} / s$ → (6)
$y \leftarrow \hat{x} * \alpha$ → (7)
$y \leftarrow y + \beta.$ → (8)

of kernal = 8



→ Method 02:-

```
def fused_layerNorm_forward(x, gamma, beta, ------):
    n_rows, n_cols = x.shape
    BLOCKS = triton.next_power_of_2(n_cols)        # Triton need Block 2^n
    if use_dlpack:
        x = torch.utils.dlpack.from_dlpack(x)
        if gamma is not None:
            gamma = torch.utils.dl____(gamma)
        if beta is not None:
            beta = torch.utils.____(beta)

        y = torch.empty_like(x)
        inv_var = torch.empty(n_rows, ---, ---)
        mean = ----------

        x_row_stride = x.stride(0)
        y_row_stride = y.stride(0)

        grid = (n_rows, )   # Block
        layerNorm_kernal_forward_traing[grid](
            y, inv_var, mean, x, gamma, beta, x_row_s,
            y_row_stride, ---, ---, Block_size).

        y = cp.from_dlpack(y)
        if traing.

            mean = cp.___(mean)
            var = ----------(-)
            return y, mean, var

        return y.
```

```
triton.jit
def layernorm_kernel_forward_training(
    output_ptr,
    inv_var_ptr, # Need for Backward Pass (N,) float32 # can be None during inference
    mean_ptr, # Need fo Backward Pass (N,) float32 # Can be None during inference
    input_ptr,
    gamma_ptr,  # 1D vector shared across all samples (E, ) can be None if no weight
    beta_ptr,   # 1D vector shared across all samples (E, ) can be None if no bias
    input_row_stride,
    output_row_stride,
    DTYPE_FLAG: tl.constexpr, # Flag for if our data is float32, float16 or bfloat16
    eps: tl.constexpr,
    n_cols: tl.constexpr, # Dimensionality of our embeddings
    BLOCK_SIZE: tl.constexpr, # closest power of 2 to our dim of embeddings
):

    """
    Operation:

    gamma * ((x - mu)/sqrt(var + eps)) + beta

    This kernel expects an (N x E) dimension matrix and so we can write an easy
    kernel that has every block normalize a single row! A more optimal method can be found
    in the Triton tutorials: https://triton-lang.org/main/getting-started/tutorials/05-layer-norm.html#sphx-glr-getting-started-tutorials-05-layer-norm-py

    But we are trying to find a balance between giving our modules some teeth, but also easily
    understandable! Im sorry to all the hardcore kernel engineers!

    The mean and variance are precision sensitive (a sum over the whole row), so just like torch
    we compute them in float32 (upcasting each element inside the kernel, no float32 copies of the
    tensor) and save mean and 1/std as float32 for the backward pass
    """

    ### Which row are we normalizing? ###
    row_idx = tl.program_id(0)

    ### Map ptrs to correct dtype ###
    pointer_dtype = tl.float32 if DTYPE_FLAG == 0 else (tl.float16 if DTYPE_FLAG == 1 else tl.bfloat16)
    output_ptr = tl.cast(output_ptr, tl.pointer_type(pointer_dtype))
    input_ptr = tl.cast(input_ptr, tl.pointer_type(pointer_dtype))
    if gamma_ptr is not None:
        gamma_ptr = tl.cast(gamma_ptr, tl.pointer_type(pointer_dtype))
    if mean_ptr is not None:
        mean_ptr = tl.cast(mean_ptr, tl.pointer_type(tl.float32))
    if inv_var_ptr is not None:
        inv_var_ptr = tl.cast(inv_var_ptr, tl.pointer_type(tl.float32))
    if beta_ptr is not None:
        beta_ptr = tl.cast(beta_ptr, tl.pointer_type(pointer_dtype))

    ### Get offsets for the full block and mask for invalid regions ###
    col_offsets = tl.arange(0, BLOCK_SIZE)
    mask = col_offsets < n_cols

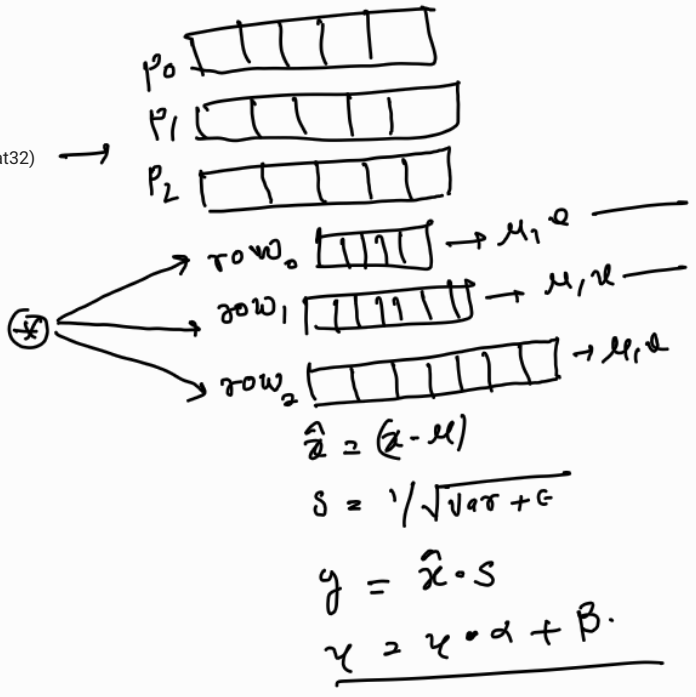
    ### Load Row (upcast to float32 for the statistics) ###
    row = tl.load(input_ptr + row_idx * input_row_stride + col_offsets, mask=mask, other=0.).to(tl.float32)

    ### Compute row mean and var w/ reduction ops ###
    row_mean = tl.sum(row, axis=0) / n_cols
    row_mean_centered = tl.where(mask, row - row_mean, 0.)
    row_var = tl.sum(row_mean_centered * row_mean_centered, axis=0) / n_cols
    inv_var = tl.rsqrt(row_var + eps)
    output = row_mean_centered * inv_var

    ### Scale and shift ###
    if gamma_ptr is not None:
        output = output * tl.load(gamma_ptr + col_offsets, mask=mask, other=0.).to(tl.float32)
    if beta_ptr is not None:
        output = output + tl.load(beta_ptr + col_offsets, mask=mask, other=0.).to(tl.float32)

    ### Write outputs (back in the input dtype) ###
    tl.store(output_ptr + row_idx * output_row_stride + col_offsets, output, mask=mask)

    ### No need to waste time writing outputs if we dont need it! ###
    if mean_ptr is not None and inv_var_ptr is not None:
        tl.store(mean_ptr + row_idx, row_mean)
        tl.store(inv_var_ptr + row_idx, inv_var)
```



```
@triton.jit
def layernorm_kernel_backward(
    x_ptr,
    gamma_ptr,
    mean_ptr,
    inv_var_ptr,
    dX_ptr,
    dGamma_ptr, # (n_blocks, E) float32 partial sums, can be None
    dB_ptr,     # (n_blocks, E) float32 partial sums, can be None
    dY_ptr,
    stride_x,
    stride_dx,
    stride_dy,
    n_rows,
    n_cols: tl.constexpr,
    BLOCK_SIZE: tl.constexpr,
    ROWS_PER_BLOCK: tl.constexpr,
    DTYPE_FLAG: tl.constexpr
):

    """
    We need to compute the gradients w.r.t to the input x. This formula is as follows:

    y = x_hat * gamma + beta
    where x_hat = (x - mu) / sqrt(var + eps)
    dL/dx = dL/dy * dy/dx_hat * dx_hat / dx

    = 1/std * [dL/dy * gamma - mean(dL/dy*gamma) - x_hat * mean(dL/dy * gamma * x_hat)]
    = 1/std * [dx_hat - mean(dx_hat) - x_hat * mean(dx_hat* x_hat)]

    dL/dgamma = sum over rows of dL/dy * x_hat
    dL/dbeta  = sum over rows of dL/dy

    Every program processes ROWS_PER_BLOCK rows at once (a 2D block). The gamma/beta grads are a sum over
    ALL rows. Accumulating them with tl.atomic_add from every program into the same (E,) buffer was over
    half the runtime (thousands of programs contending on the same E cells), so instead each program
    writes the partial sum for its own block of rows into its own row of an (n_blocks, E) buffer, and the
    caller does a single .sum(0) at the end.
    """

    block_idx = tl.program_id(0)

    ### Map Pointers To Correct Dtype ###
    pointer_type = tl.float32 if DTYPE_FLAG == 0 else (tl.float16 if DTYPE_FLAG == 1 else tl.bfloat16)
    x_ptr = tl.cast(x_ptr, tl.pointer_type(pointer_type))
    mean_ptr = tl.cast(mean_ptr, tl.pointer_type(tl.float32))
    inv_var_ptr = tl.cast(inv_var_ptr, tl.pointer_type(tl.float32))
    dX_ptr = tl.cast(dX_ptr, tl.pointer_type(pointer_type))
    dY_ptr = tl.cast(dY_ptr, tl.pointer_type(pointer_type))
    if gamma_ptr is not None:
        gamma_ptr = tl.cast(gamma_ptr, tl.pointer_type(pointer_type))
    if dGamma_ptr is not None:
        dGamma_ptr = tl.cast(dGamma_ptr, tl.pointer_type(tl.float32))
    if dB_ptr is not None:
        dB_ptr = tl.cast(dB_ptr, tl.pointer_type(tl.float32))

    ### Rows (R,) and columns (E,) this program handles ###
    rows = block_idx * ROWS_PER_BLOCK + tl.arange(0, ROWS_PER_BLOCK)
    row_mask = rows < n_rows
    cols = tl.arange(0, BLOCK_SIZE)
    col_mask = cols < n_cols
    mask = row_mask[:, None] & col_mask[None, :]

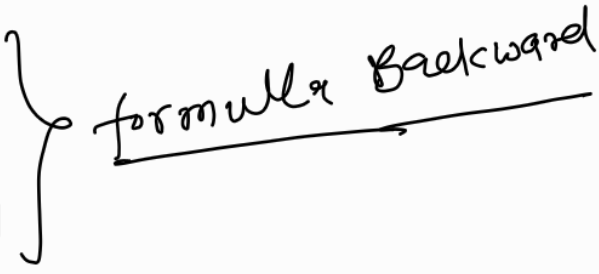
    ### Load all the data (R, E) in float32 ###
    x = tl.load(x_ptr + rows[:, None] * stride_x + cols[None, :], mask=mask, other=0.0).to(tl.float32)
    dy = tl.load(dY_ptr + rows[:, None] * stride_dy + cols[None, :], mask=mask, other=0.0).to(tl.float32)
    row_mean = tl.load(mean_ptr + rows, mask=row_mask, other=0.0)[:, None]
    row_inv_std = tl.load(inv_var_ptr + rows, mask=row_mask, other=0.0)[:, None]

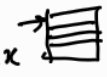
    if gamma_ptr is not None:
        gamma = tl.load(gamma_ptr + cols, mask=col_mask, other=0.).to(tl.float32)[None, :]
    else:
        gamma = 1.0

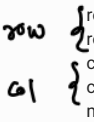
    ### Compute backward pass ###
    x_hat = (x - row_mean) * row_inv_std
    wdy = gamma * dy # <- upstream grad (dL/dy)(dy/dx_hat)
    mean1 = tl.sum(x_hat * wdy, axis=1)[:, None] / n_cols
    mean2 = tl.sum(wdy, axis=1)[:, None] / n_cols
    dx = row_inv_std * (wdy - (x_hat * mean1 + mean2))

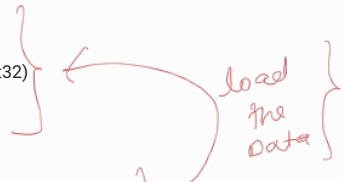
    ### Store dx ###
    tl.store(dX_ptr + rows[:, None] * stride_dx + cols[None, :], dx, mask=mask)

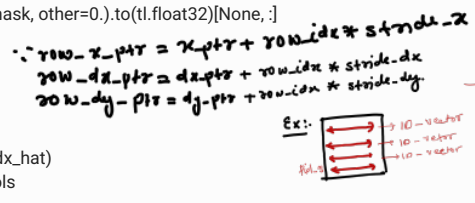
    ### Partial sums of the weight/bias grads for this block of rows (kept in float32) ###
    if dGamma_ptr is not None:
        tl.store(dGamma_ptr + block_idx * n_cols + cols, tl.sum(dy * x_hat, axis=0), mask=col_mask)
    if dB_ptr is not None:
        tl.store(dB_ptr + block_idx * n_cols + cols, tl.sum(dy, axis=0), mask=col_mask)
```

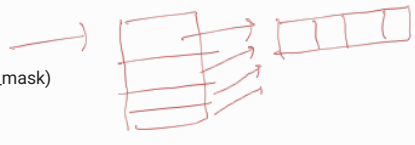












→ Method 03 :-

forward pass

```
def fused_layernorm_forward(x, gamma=None, beta=None, eps=1e-5, training=True, use_dlpack=True):
    """
    x: Input (N, E)
    gamma: scale parameter (E,)
    beta: shift parameter (E,)

    Returns:
        y: Ouptut (N, E)
        mean: per row mean (N,) float32
        inv_var: per row 1/std (N,) float32
    """

    n_rows, n_cols = x.shape
    BLOCK_SIZE = triton.next_power_of_2(n_cols)
    dtype_flag = get_dtype_flag(x.dtype)  # 0=float32, 1=float16, 2=bfloat16
    grid = (n_rows,)

    if not DLPACK_DISABLE and use_dlpack:

        x = torch.utils.dlpack.from_dlpack(x)
        if gamma is not None:
            gamma = torch.utils.dlpack.from_dlpack(gamma)
        if beta is not None:
            beta = torch.utils.dlpack.from_dlpack(beta)

        # Allocate outputs (saved statistics are always float32)
        y = torch.empty_like(x)
        inv_var = torch.empty(n_rows, dtype=torch.float32, device=x.device) if training else None
        mean = torch.empty(n_rows, dtype=torch.float32, device=x.device) if training else None

        layernorm_kernel_forward_training[grid](
            y, inv_var, mean, x, gamma, beta,
            x.stride(0), y.stride(0),
            dtype_flag, eps, n_cols, BLOCK_SIZE,
        )

        y = cp.from_dlpack(y)
        if training:
            return y, cp.from_dlpack(mean), cp.from_dlpack(inv_var)
        return y

    else:
        # Allocate outputs (saved statistics are always float32)
        y = cp.empty_like(x)
        mean = cp.empty((n_rows,), dtype=cp.float32) if training else None
        inv_var = cp.empty((n_rows,), dtype=cp.float32) if training else None

        layernorm_kernel_forward_training[grid](
            TritonPtr(y),
            TritonPtr(inv_var) if inv_var is not None else None,
            TritonPtr(mean) if mean is not None else None,
            TritonPtr(x),
            TritonPtr(gamma) if gamma is not None else None,
            TritonPtr(beta) if beta is not None else None,
            x.strides[0] // x.itemsize, y.strides[0] // y.itemsize,
            dtype_flag, eps, n_cols, BLOCK_SIZE,
        )

        if training:
            return y, mean, inv_var
        return y
```

Backward pass :-

```
def fused_layernorm_backward(x, mean, inv_var, dy, gamma=None, beta=None, use_dlpack=True):
    n_rows, n_cols = dy.shape
    BLOCK_SIZE = triton.next_power_of_2(n_cols)
    ROWS_PER_BLOCK = _rows_per_block(BLOCK_SIZE)
    n_blocks = triton.cdiv(n_rows, ROWS_PER_BLOCK)
    dtype_flag = get_dtype_flag(dy.dtype)
    grid = (n_blocks,)

    gamma_dtype = gamma.dtype if gamma is not None else None
    beta_dtype = beta.dtype if beta is not None else None

    if not DLPACK_DISABLE and use_dlpack:
        x = torch.utils.dlpack.from_dlpack(x)
        dy = torch.utils.dlpack.from_dlpack(dy)
        mean = torch.utils.dlpack.from_dlpack(mean)
        inv_var = torch.utils.dlpack.from_dlpack(inv_var)
        gamma = torch.utils.dlpack.from_dlpack(gamma) if gamma is not None else None

        dx = torch.empty_like(dy)
        dgamma_blocks = torch.empty((n_blocks, n_cols), dtype=torch.float32, device=dy.device) if gamma is not None else None
        dbeta_blocks = torch.empty((n_blocks, n_cols), dtype=torch.float32, device=dy.device) if beta is not None else None

        layernorm_kernel_backward[grid](
            x, gamma, mean, inv_var, dx, dgamma_blocks, dbeta_blocks, dy,
            x.stride(0), dx.stride(0), dy.stride(0), n_rows, n_cols, BLOCK_SIZE, ROWS_PER_BLOCK, dtype_flag,
        )

        dx = cp.from_dlpack(dx)
        dgamma = cp.from_dlpack(dgamma_blocks.sum(0)).astype(gamma_dtype, copy=False) if gamma is not None else None
        dbeta = cp.from_dlpack(dbeta_blocks.sum(0)).astype(beta_dtype, copy=False) if beta is not None else None

    else:
        dx = cp.empty_like(dy)
        dgamma_blocks = cp.empty((n_blocks, n_cols), dtype=cp.float32) if gamma is not None else None
        dbeta_blocks = cp.empty((n_blocks, n_cols), dtype=cp.float32) if beta is not None else None

        layernorm_kernel_backward[grid](
            TritonPtr(x),
            TritonPtr(gamma) if gamma is not None else None,
            TritonPtr(mean), TritonPtr(inv_var), TritonPtr(dx),
            TritonPtr(dgamma_blocks) if gamma is not None else None,
            TritonPtr(dbeta_blocks) if beta is not None else None,
            TritonPtr(dy),
            x.strides[0] // x.itemsize, dx.strides[0] // dx.itemsize, dy.strides[0] // dy.itemsize,
            n_rows, n_cols, BLOCK_SIZE, ROWS_PER_BLOCK, dtype_flag,
        )

        dgamma = dgamma_blocks.sum(0).astype(gamma_dtype, copy=False) if gamma is not None else None
        dbeta = dbeta_blocks.sum(0).astype(beta_dtype, copy=False) if beta is not None else None

    return dx, dgamma, dbeta
```